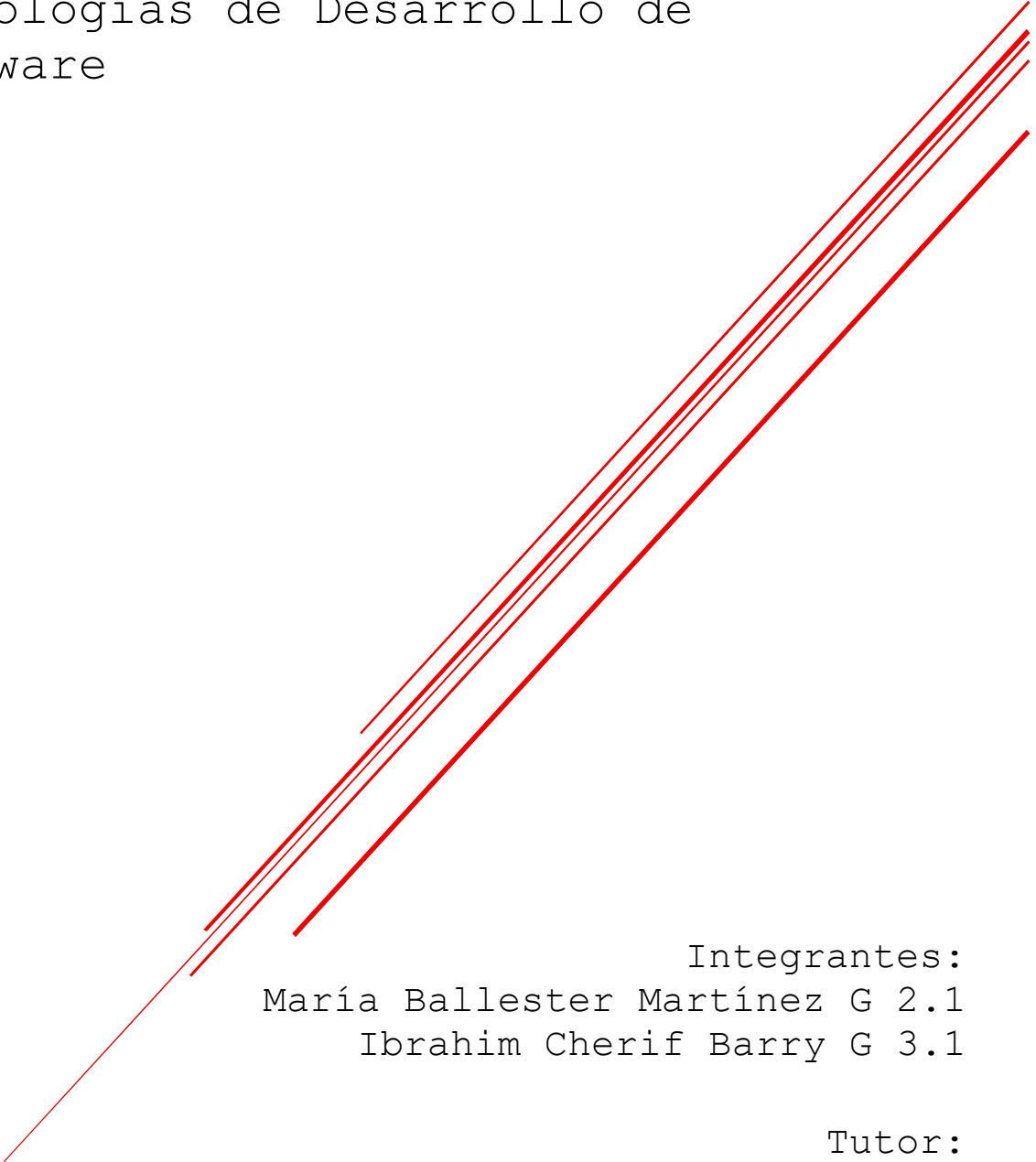


MEMORIA APPCHAT

Tecnologías de Desarrollo de
Software



Integrantes:
María Ballester Martínez G 2.1
Ibrahim Cherif Barry G 3.1

Tutor:
Jesús Sánchez Cuadrado

Índice

1.	Historias de usuario.....	3
2.	Diagramas de clases y de secuencia.....	6
2.1.	Diagrama de clases inicial de María	6
2.2.	Diagrama de clases inicial de Ibrahim	6
2.3.	Diagrama de clases finalmente empleado.....	7
2.4.	Diagrama de secuencia	8
3.	Arquitectura de la aplicación y decisiones de diseño	9
4.	Patrones de diseño utilizados	10
5.	Manual de usuario.....	12
5.1.	Iniciar aplicación.....	12
5.2.	Iniciar sesión	12
5.3.	Registrarse.....	13
5.4.	Cerrar sesión	14
5.5.	Cambiar la imagen de perfil.....	15
5.6.	Cambiar el saludo	16
5.7.	Añadir contacto	17
5.8.	Eliminar contacto	18
5.9.	Crear grupo	18
5.10.	Eliminar grupo	19
5.11.	Añadir/Eliminar Miembro de un grupo	19
5.12.	Crear los PDFs	20
5.13.	Buscar mensajes.....	22
5.14.	Activación del premium.....	23
6.	Conclusiones	24

Índice de Ilustraciones

Ilustración 1: Diagrama de clases de María	6
Ilustración 2: Diagrama de clases de Ibrahim	6
Ilustración 3: Diagrama de clases final	7
Ilustración 4: Diagrama de secuencia	8
Ilustración 5: Diagrama de clases del patrón Composite en el manejo de filtros	10
Ilustración 6: Ventana de login	13
Ilustración 7: Pestaña para seleccionar una imagen de perfil	13
Ilustración 8: Ventana de registro	14
Ilustración 9: Ventana principal	15
Ilustración 10: Darle a "cambiar imagen de perfil"	15
Ilustración 11: Introducir URL de imagen deseada	15
Ilustración 12: Notificación de imagen modificada	16
Ilustración 13: Imagen modificada	16
Ilustración 14: Seleccionar "cambiar saludo"	16
Ilustración 15: Introduciendo saludo	16
Ilustración 16: Confirmación de saludo modificado	16
Ilustración 17: Ventana para gestionar los contactos individuales	17
Ilustración 18: Pestaña para insertar los datos de un nuevo contacto	17
Ilustración 19: Ventana principal con ejemplo de contacto sin agregar	18
Ilustración 20: Ventana para gestionar los grupos	18
Ilustración 21: Pestaña nombre del grupo	18
Ilustración 22: Añadir miembros a un grupo	19
Ilustración 23: Eliminar miembros de un grupo	19
Ilustración 24: Ejemplo PDF Listado generado	20
Ilustración 25: Ejemplo PDF Listado generado	21
Ilustración 26: Ventana para buscar	22
Ilustración 27: Ventana buscar funcionando	22
Ilustración 28: Pregunta por si confirmas activar premium	23
Ilustración 29: Notificación tras activar premium	23

1. Historias de usuario

Historia de usuario 1: “Iniciar sesión”

Como usuario, **quiero** iniciar sesión **para** enviar y recibir mensajes.

Criterios de aceptación:

1. Criterio para iniciar sesión un usuario registrado

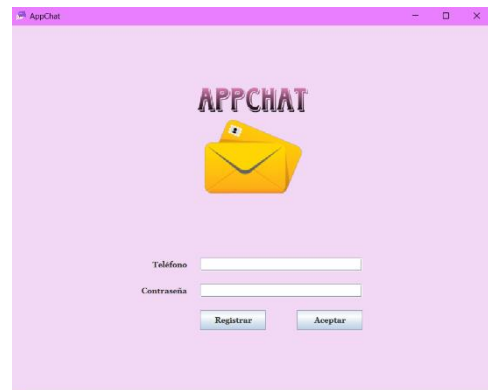
Dado que el usuario está en la ventana de inicio de la aplicación, **cundo** el usuario introduzca la información en los campos (número de teléfono y contraseña) y le dé al botón “Aceptar”, **entonces** el sistema debe mostrarle la ventana principal.

2. Criterio para iniciar sesión un usuario no registrado

Dado que el usuario está en la ventana de inicio de la aplicación, **cundo** el usuario introduzca la información en los campos (número de teléfono y contraseña) y le dé al botón “Aceptar”, pero previamente no se haya registrado; **entonces** el sistema debe volver a mostrarle la ventana login y con un mensaje de error que ponga “Usuario no registrado”.

3. Criterio para iniciar sesión de un usuario registrado previamente que falla su contraseña

Dado que el usuario está en la ventana de inicio de la aplicación, **cundo** el usuario introduzca la información en los campos (número de teléfono y contraseña) y le dé al botón “Aceptar”, pero previamente no sea la contraseña correcta; **entonces** el sistema debe volver a mostrarle la ventana login y con un mensaje de error que ponga “Contraseña incorrecta”.



Historia de usuario 2: “Registrar”

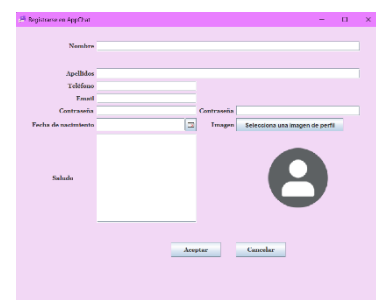
Como usuario, **quiero** registrarme, **para** poder enviar y/o recibir mensajes.

Criterio de aceptación:

1. Criterio de registro de usuario

Dado que el usuario está en la ventana login, **cundo** selecciona el botón de registrar, **entonces** el sistema debe mostrar los campos necesarios para completar el registro del usuario en la aplicación.

2. Criterio de validación de registro



Dado que el usuario está en la ventana registro, **cundo** el usuario no haya introducido los campos obligatorios (todos menos el saludo y la imagen), **entonces** el sistema debe mostrar un mensaje de error indicando el error y permitir al usuario intentarlo otra vez.

3. Criterio de validación de contraseña

Dado que el usuario está en la ventana registro, **cuando** las contraseñas no coincidan, **entonces** el sistema debe mostrar un mensaje de error indicando el error y permitir al usuario intentarlo otra vez.

Historia de usuario 3: “Añadir contacto”

Como usuario, **quiero** añadir un número de teléfono a mis contactos **para** agregarlo a la lista de contactos y poder enviarle mensajes y crear grupos con él.

Criterios de aceptación:

1. Criterio de añadir a contactos un número de teléfono que te haya enviado un mensaje con anterioridad

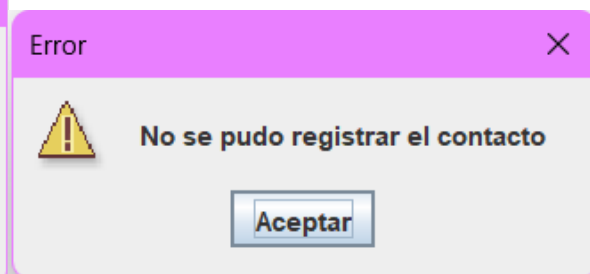
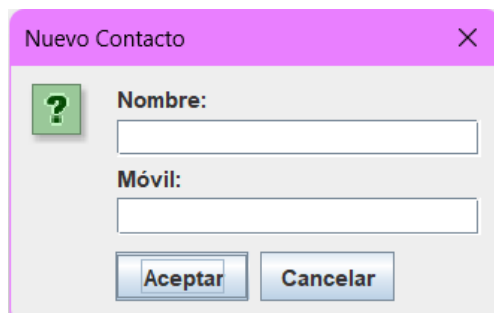
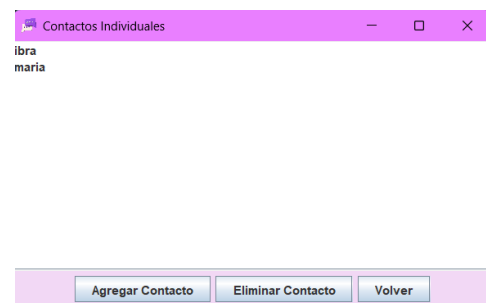
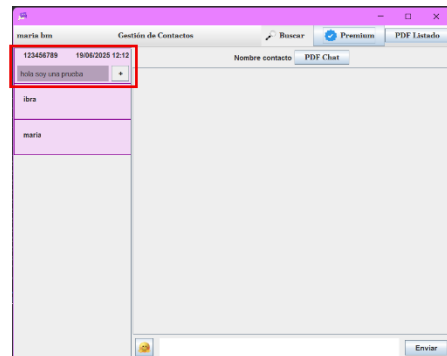
Dado que el usuario está en la ventana principal de la aplicación, **cuando** presione el botón “+” al lado de un mensaje de un número no agregado, **entonces** el sistema debe mostrar la pestaña de añadir contacto con el campo de teléfono autocompletado con el número asociado al botón “+”.

2. Criterio de añadir a la lista de contactos un número existente

Dado que el usuario está en la ventana de Añadir contacto de la aplicación, **cuando** complete los campos mostrados (nombre y teléfono), y además pulse el botón “Aceptar”, **entonces** el sistema debe añadir a la lista de contactos el nuevo contacto creado.

3. Criterio de añadir a la lista de contactos un número no existente

Dado que el usuario está en la ventana de Añadir contacto de la aplicación, **cuando** complete los campos mostrados (nombre y teléfono), y pulse el botón “Aceptar”, pero el número de teléfono no sea de uno de los usuarios registrados en AppChat, **entonces** el sistema debe mostrar una pestaña de error al añadir el contacto, con un mensaje que diga: “No se pudo registrar el contacto”.



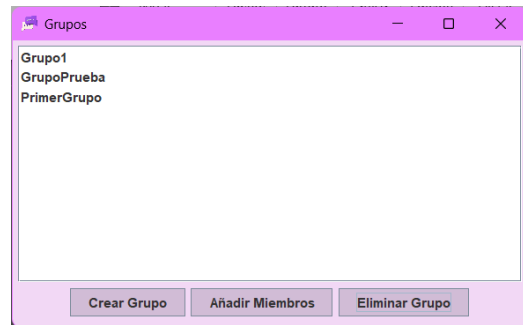
Historia de usuario 4: “Crear grupo”

Como usuario, **quiero** crear grupo, **para** poder enviar mensajes a un grupo de contactos.

Criterio de aceptación:

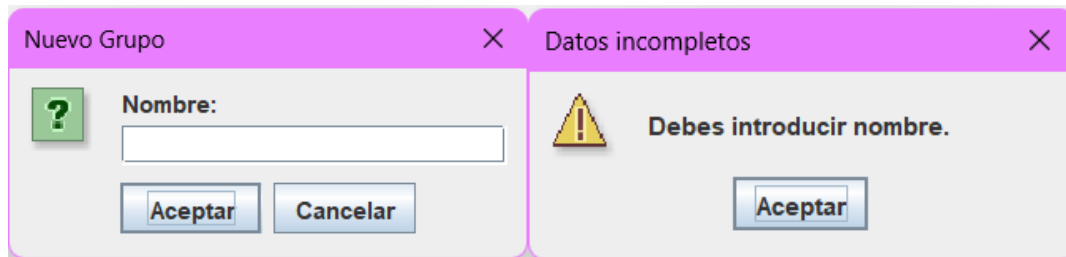
1. Criterio de creación de grupos

Dado que el usuario está en la ventana contactos, **cundo** selecciona el botón de añadir grupo, **entonces** el sistema debe mostrar los campos necesarios para completar la creación del grupo, nombre y opcionalmente una imagen, y añadir los contactos.



2. Criterio de creación de grupos cuando el usuario no complete algún campo obligatorio

Dado que el usuario está en la ventana de Añadir Grupo, **cundo** no complete todos los campos necesarios para la creación del grupo y le dé al botón “Aceptar”, **entonces** el sistema debe mostrar un error que diga que campo falta por completar (solo puede ser el nombre).



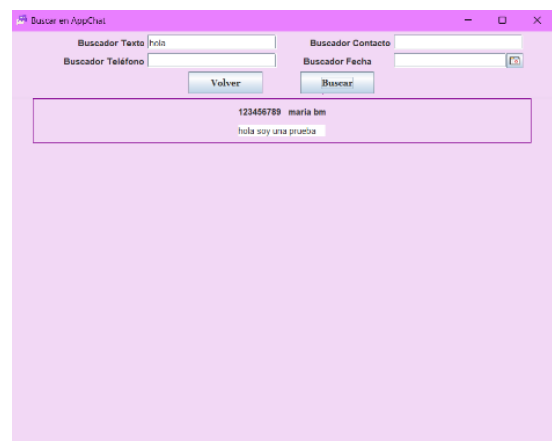
Historia de usuario 5: “Aplicar filtros”

Como usuario, **quiero** aplicar los filtros seleccionados, **para** buscar mensajes.

Criterio de aceptación:

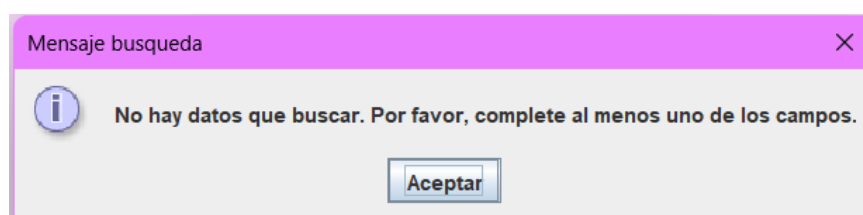
1. Criterio de aplicar filtros para mensajes existentes

Dado que el usuario está en la ventana buscar, **cundo** complete los campos de los diferentes filtros, **entonces** el sistema debe mostrar los mensajes coincidentes del emisor y/o del receptor.



2. Criterio para aplicar filtros sin completar ningún filtro

Dado que el usuario está en la ventana buscar, **cundo** presione el botón “Buscar” sin haber completado ningún campo de los filtros, **entonces** el sistema debe mostrar un mensaje avisándole con el mensaje “No hay datos que buscar. Por favor, complete al menos uno de los campos.”.



2. Diagramas de clases y de secuencia

2.1. Diagrama de clases inicial de María

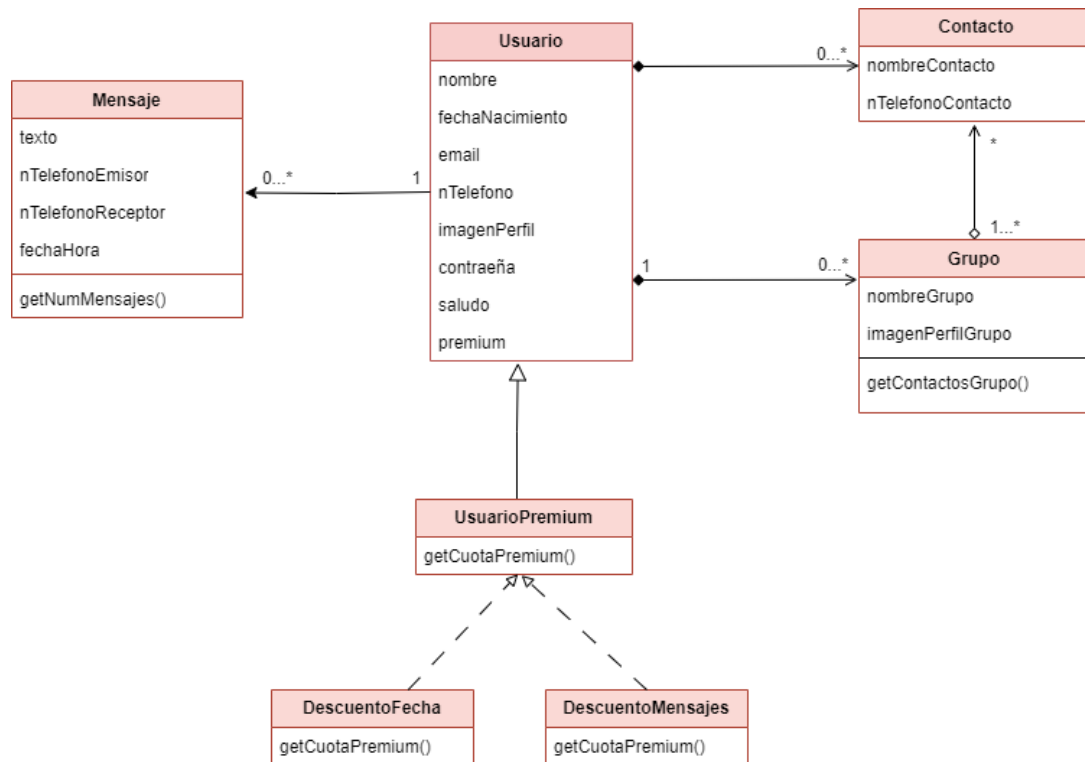


Ilustración 1: Diagrama de clases de María

2.2. Diagrama de clases inicial de Ibrahim

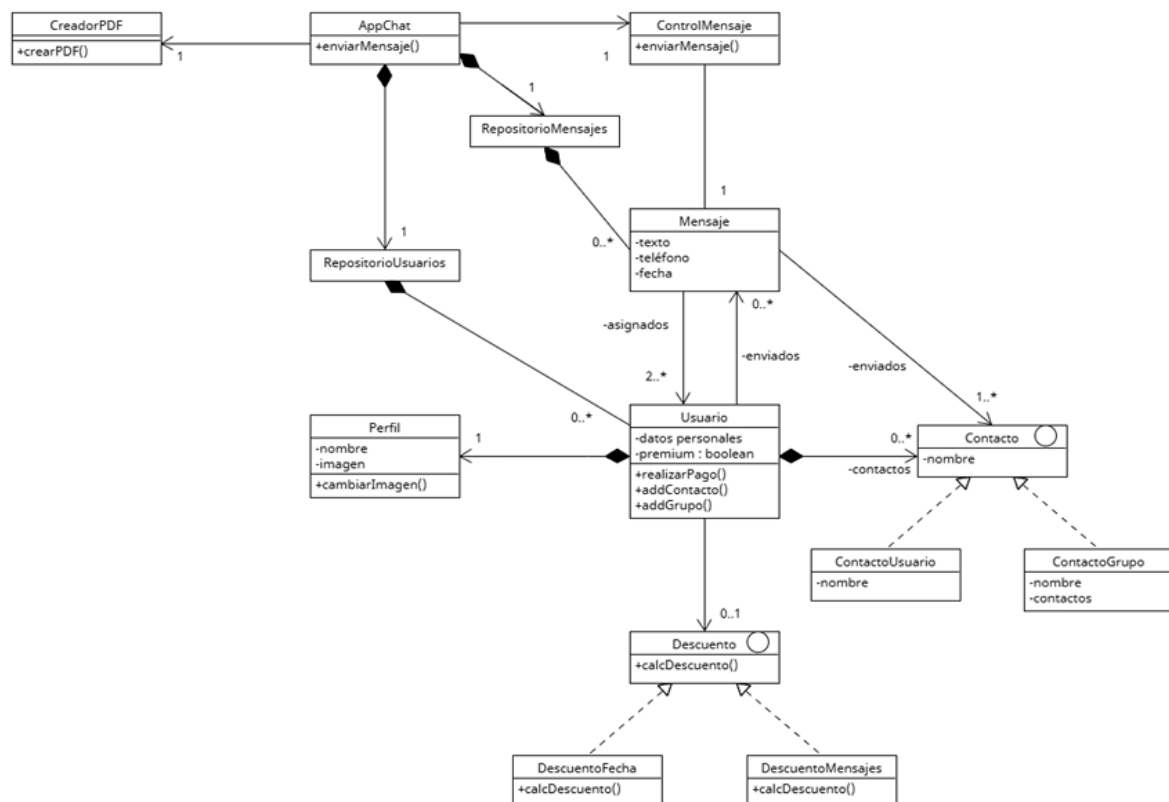


Ilustración 2: Diagrama de clases de Ibrahim

2.3. Diagrama de clases finalmente empleado

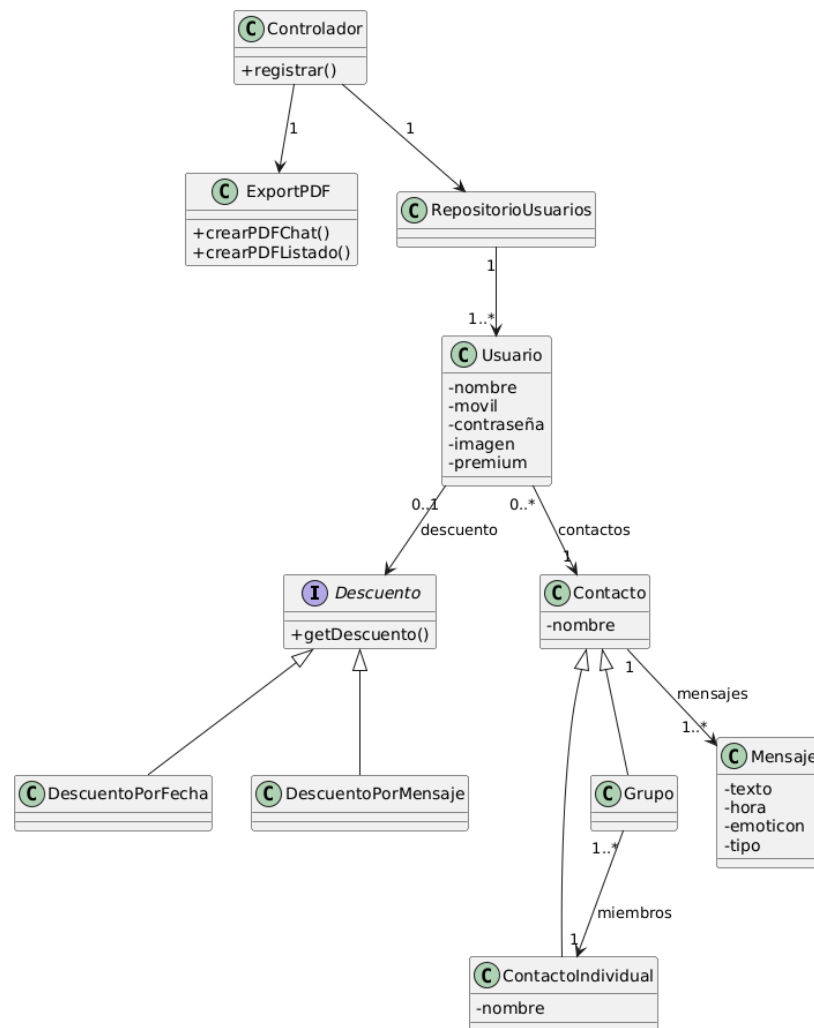


Ilustración 3: Diagrama de clases final

La clase **Controlador** se encarga, entre muchas otras cosas, de registrar usuarios, y coordinar la lógica de la aplicación. Se relaciona con **ExportPDF** (que permite generar PDFs de chats y listados) y con **RepositorioUsuarios**, que administra múltiples instancias de la clase **Usuario**. Cada **Usuario** posee atributos como nombre, móvil, contraseña, imagen y si es premium, y puede tener múltiples **Contactos**. Estos contactos pueden ser individuales (**ContactoIndividual**) o pertenecer a un **Grupo**, y se asocian con múltiples **Mensajes** que incluyen texto, hora, emoticono y tipo. Además, cada usuario puede tener asociado un **Descuento**, que implementa el método `getDescuento()` y puede ser de tipo **DescuentoPorFecha** o **DescuentoPorMensaje**.

2.4. Diagrama de secuencia

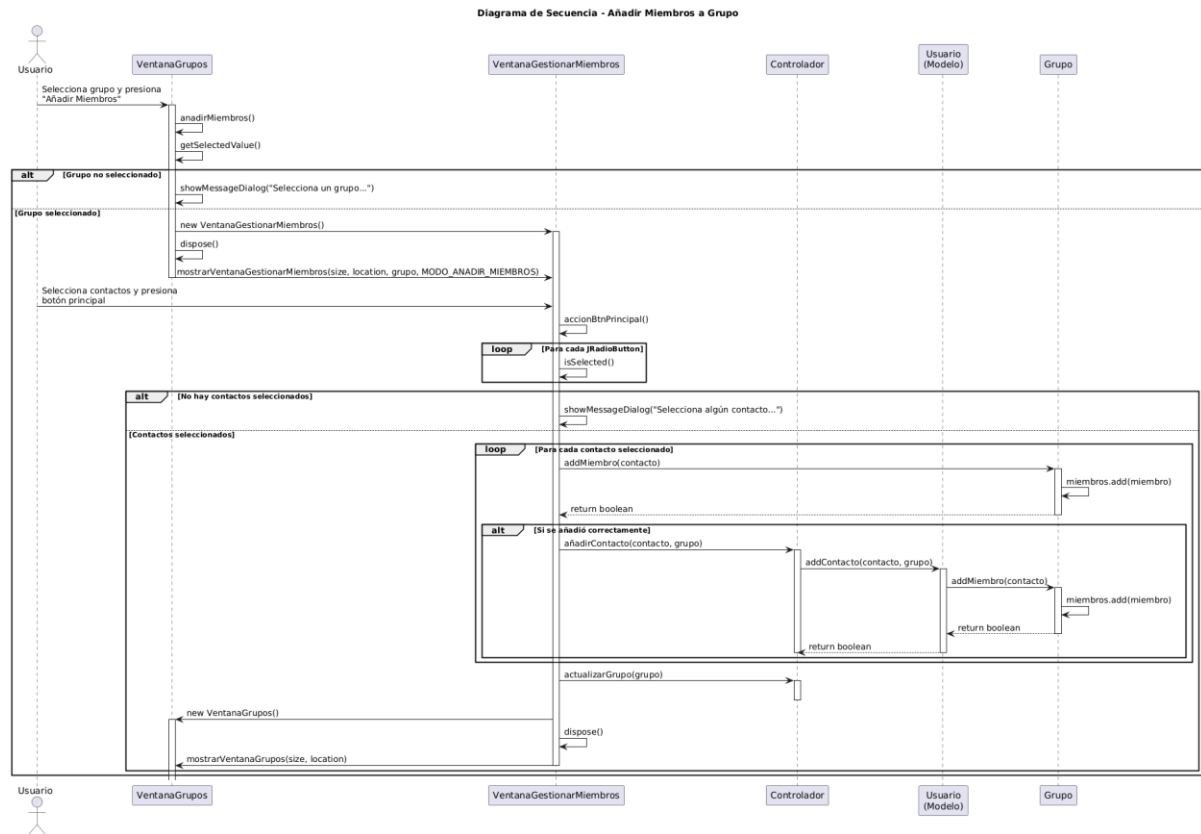


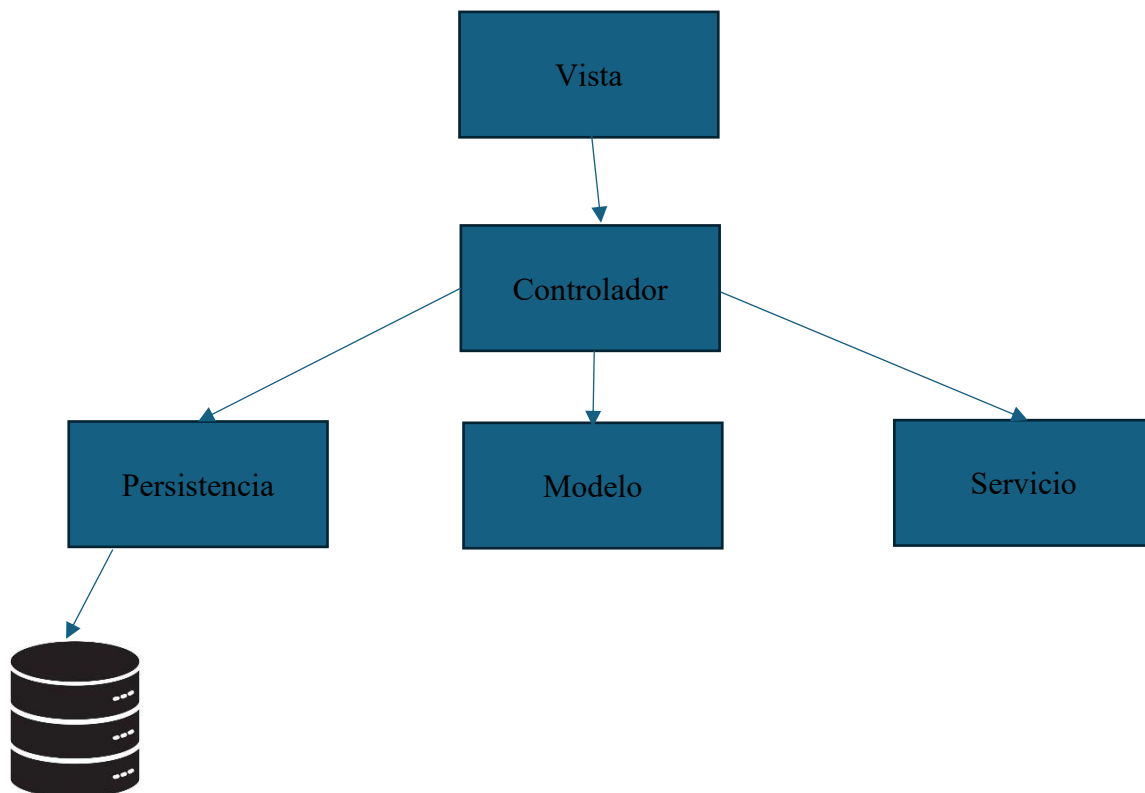
Ilustración 4: Diagrama de secuencia

El diagrama de secuencia ilustra una implementación del patrón arquitectónico Modelo-Vista-Controlador (MVC). La Vista está representada por las clases *VentanaGrupos* y *VentanaGestionarMiembros*, que se encargan exclusivamente de la presentación y captura de eventos del usuario, sin contener lógica de negocio. El Controlador actúa como mediador entre las vistas y el modelo, recibiendo las peticiones de la interfaz gráfica y coordinando las operaciones necesarias en el modelo de datos. El Modelo está compuesto por las entidades *Usuario* y *Grupo*, que encapsulan tanto los datos como las reglas de negocio de la aplicación. Este diseño respeta los principios fundamentales del MVC: separación de responsabilidades, bajo acoplamiento entre capas, y mantenimiento de la integridad del modelo de datos. La secuencia demuestra cómo las vistas nunca acceden directamente al modelo, sino que todas las modificaciones de estado se canalizan a través del controlador, garantizando así un flujo de datos unidireccional y controlado que facilita el mantenimiento y la escalabilidad del sistema.

3. Arquitectura de la aplicación y decisiones de diseño

Hemos tratado de ceñirnos al máximo en lo que viene a ser una arquitectura software basada en capas, aplicando el principio fundamental de Separación Modelo-Vista.

En nuestra arquitectura, la Vista recibe las peticiones del usuario, se las comunica al Controlador, y este último delega las peticiones de la Vista a la Persistencia (si implica persistir, modificar, eliminar o recuperar entidades de la base de datos), al Modelo (si implica manejar entidades del dominio en memoria), o a un determinado Servicio concreto (así como un generador de PDFs).



Base de datos

En lo que respecta a las decisiones de diseño, para garantizar la modularidad del sistema, creímos conveniente dividirlo en varios paquetes y subpaquetes:

- umu.tds.apps.aplicacion: en este paquete se encuentran el punto de entrada para ejecutar AppChat, y un cargador de entidades para ejecutar la aplicación con ciertos usuarios, contactos, grupos, y mensajes.
- umu.tds.apps.controlador: paquete donde se encuentra el Controlador de la aplicación.
- umu.tds.apps.dominio: en este paquete están todas las entidades del dominio.
- umu.tds.apps.dominio.descuentos: subpaquete del dominio, donde se encuentra todo lo relacionado con las políticas de descuento.
- umu.tds.apps.persistencia: paquete donde está toda la lógica de los DAOs.

- umu.tds.apps.persistencia.tdsimpl: subpaquete de la persistencia, donde se encuentran los DAOs implementados para usar el servicio de persistencia de TDS.
- umu.tds.apps.repositorios: en este paquete se encuentra un repositorio de usuarios.
- umu.tds.apps.servicios: aquí se encuentran aquellos servicios externos en los que mayoritariamente el Controlador delega ciertas tareas específicas.
- umu.tds.apps.servicios.descargas: en este subpaquete de servicios se encuentra la gestión de obtención de rutas de descargas independientemente del sistema operativo.
- umu.tds.apps.servicios.filtros: en este subpaquete de servicios se encuentra toda la lógica de filtrado de mensajes.
- umu.tds.apps.utils: paquete con una clase de utilidades con ciertas constantes.
- umu.tds.apps.vista: en este paquete se encuentra lo que viene a ser el lado frontend, la que ve el usuario final.

4. Patrones de diseño utilizados

Patrones de diseño usados directamente

- Composite: empleamos este patrón en el manejo de filtros de mensajes. Un filtro compuesto que pueda acumular dinámicamente varios filtros base, para aplicarlos todos directamente.

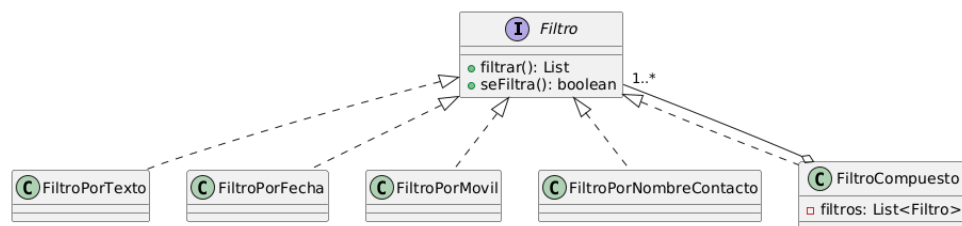


Ilustración 5: Diagrama de clases del patrón Composite en el manejo de filtros

- Estrategia: patrón aplicado para la política de Descuentos hacia un Usuario. Definimos una familia de distintos algoritmos de aplicación de Descuento, y en función de las condiciones de un Usuario, nos quedamos con una única política de descuento. Este patrón viene acompañado de una Factoría.
- Método Factoría: nuestro servicio PDF necesita recuperar la carpeta de Descargas del ordenador donde se ejecuta AppChat, independientemente del sistema operativo. Para ello, disponemos de diferentes proveedores de rutas de descargas, una por sistema operativo (Linux, Windows, MAC). Y se delega a una Factoría que, en función del sistema operativo donde se esté ejecutando la aplicación, obtenga la ruta de descargas correspondiente.
- Fachada: dada la arquitectura software empleada, conseguimos que el Controlador actúe como un intermediario entre la Vista y el Modelo.
- Singleton: garantizamos un único Controlador que coordine toda la lógica de la aplicación, un único Servicio que maneje la generación de PDFs, y un único servicio para cada adaptador de los DAOs.

Patrones de diseño usados indirectamente

- Composite: para la creación de Menús con JMenu y JMenuItem.
- DAO: Factoría Abstracta (crear diferentes familias de DAOs, como TDSFactoriaDAO), y Adaptador (para cada EntidadDAO, que soporte diferentes servicios de persistencia externos, como puede ser el Servicio de Persistencia de TDS).
- Observer: registrar en elementos de la vista como botones, un objeto listener (el cual implementa la interfaz ActionListener) para escuchar los eventos de acción que ocurren en ese elemento de la vista.
- Método Factoría: uso de fuentes tipográficas con la librería iText.

5. Manual de usuario

5.1. Iniciar aplicación

Asegúrese de tener instalado Maven en su ordenador. Es imprescindible para poder instalar las dependencias, como son la librería chatWindowLib y el Driver de Persistencia. Una vez tenga instalado Maven, en primer lugar debe acceder a la carpeta donde se encuentran chatWindowLib.jar (carpeta AppChat) ejecutar desde el terminal:

```
mvn install:install-file -Dfile=chatWindowLib.jar -DgroupId=tds  
-DartifactId=chat-window -Dversion="1.0.0" -Dpackaging=jar  
-DgeneratePom=true
```

Y luego, ejecutar lo siguiente desde la carpeta donde tengas DriverPersistencia.jar (carpeta lib):

```
mvn install:install-file -Dfile=DriverPersistencia.jar  
-DpomFile=driverPersistencia-2.0.pom
```

Así, tendrá todas las dependencias necesarias para la ejecución de la aplicación, ya que se instalarán en el repositorio local de Maven.

AppChat es una aplicación de escritorio desarrollada con Java/Swing que para poder ejecutarla hay que poner en la terminal la instrucción `java -jar .\ServidorPersistenciaH2.jar`. Este paso debe realizarse desde una terminal ubicada en la ruta donde se encuentra el archivo `ServidorPersistenciaH2.jar`.

Después de iniciar el servidor de persistencia que da acceso a la base de datos, tenemos que ejecutar el código de la clase `AppChat.java` que se encuentra en la carpeta `"AppChat\src\main\java\umu\tds\apps\aplicacion\"`.

5.2. Iniciar sesión

Nada más iniciar la aplicación se mostrará la pestaña que se ve en la Ilustración 6. Para acceder a su cuenta se debe completar los campos de un formulario con el que el sistema podrá comprobar la autenticidad del usuario que está intentado iniciar sesión. Los campos que se deben completar son:

- **Teléfono:** en este campo se debe poner el número de teléfono asociado a la cuenta de usuario al que se quiere acceder.
- **Contraseña:** en este campo se debe poner la contraseña asociada al usuario del teléfono anterior.

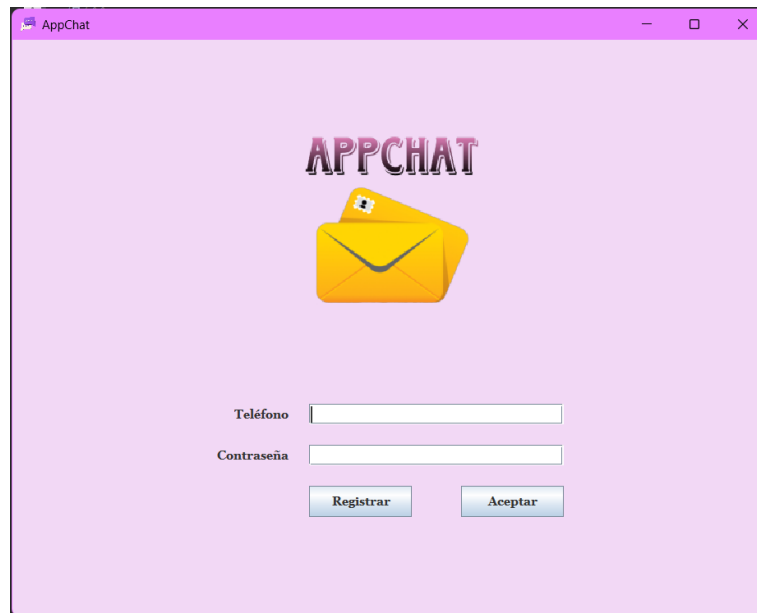


Ilustración 6: Ventana de login

5.3. Registrarse

La primera vez que se inicie la aplicación el usuario no tendrá una cuenta y se deberá crear una. Para crearla, en la ventana que se muestra en la Ilustración 6 hay que presionar el botón con el rótulo “Registrar”. Cuando ese botón se presiona se muestra la ventana de registro, que es la que se muestra en la Ilustración 8. Esta ventana también muestra un formulario, pero con muchos más campos que en la ventana anterior. Estos campos son:

- Nombre: campo donde se debe ingresar el nombre de la persona que se quiere registrar.
- Apellidos: campo donde se debe ingresar el apellido de la persona que se quiere registrar.
- Teléfono: en este campo se debe introducir un numero al que asociar el nuevo usuario.
- Email: campo donde se debe ingresar el email de la persona que se quiere registrar.
- Contraseña: hay dos campos en los que se tienen que escribir la contraseña con la que se pretende iniciar sesión.
- Fecha de nacimiento: en el icono del calendario en ese mismo campo se debe seleccionar el día, mes y año.
- Saludo: este es un campo donde se puede escribir unas frases de presentación.
- Imagen: en el botón “Selecciona una imagen de perfil”, se abre una pestaña en la que se puede introducir una url. Esta pestaña es la que encontramos en la Ilustración 7.

De todos estos campos no todos son obligatorios, pero la gran mayoría sí. Para simplificar, se indicarán únicamente los campos opcionales: en este caso, *Saludo* e *Imagen*.

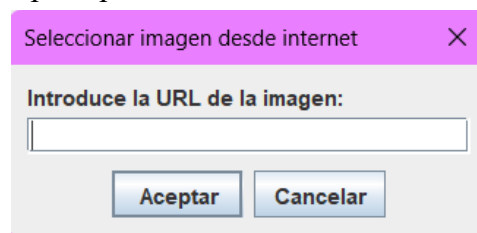


Ilustración 7: Pestaña para seleccionar una imagen de perfil

Para la mayoría de los campos mencionados encontramos ciertas restricciones. Entre ellas están:

- Teléfono: este no debe estar registrado ya en la base de datos de la aplicación y debe ser de 9 dígitos.
- Email: el contenido del campo debe tener un formato. Debe comenzar con un nombre de usuario (letras, números, guiones o puntos), incluir el símbolo @, tener un nombre de dominio (como gmail, empresa, etc.) y finalizar con una extensión de dominio (como .com, .es, .org), con al menos dos letras. Un ejemplo es: usuario@dominio.com
- Contraseña: los campos de las contraseñas deben coincidir.
- Fecha de nacimiento: la fecha no puede ser futura a la creación de la cuenta.

Tras completar todos los campos hay que darle al botón “Aceptar”. Si alguno de los requisitos descritos anteriormente o algún campo obligatorio no este completo saltara un mensaje de error indicando el motivo.

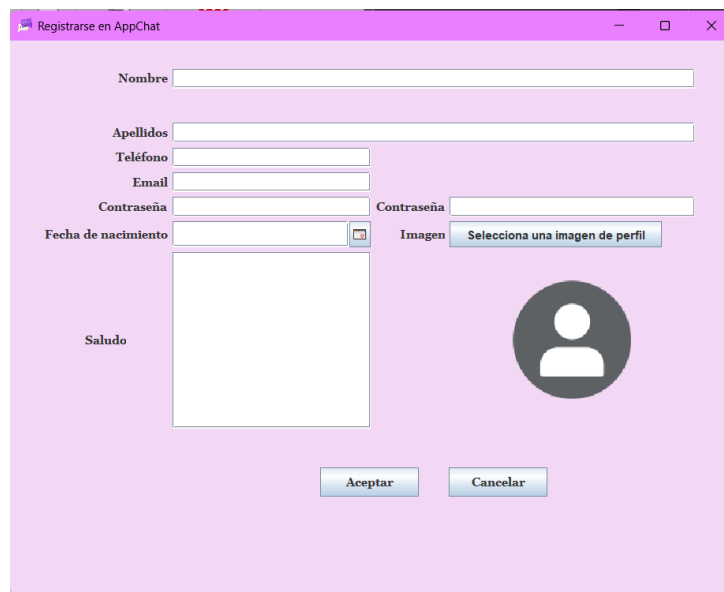
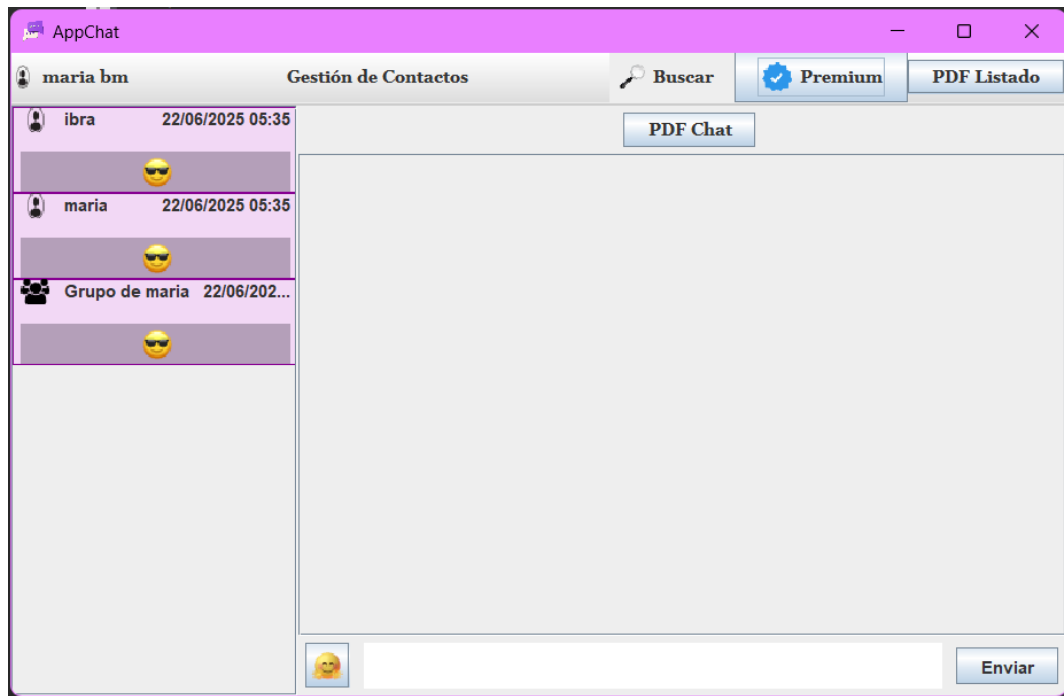


Ilustración 8: Ventana de registro

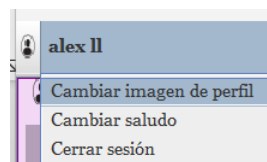
5.4. Cerrar sesión

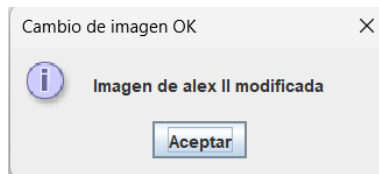
Una vez que se ha terminado de usar los servicios de la aplicación y se desea cerrar sesión solo hay que presionar en el menú superior, de la ventana principal mostrada en la Ilustración 9, su nombre de perfil. Esto desplegará un pequeño menú desplegable en el que si se selecciona la opción “Cerrar sesión” se volverá a la pantalla para hacer login.

*Ilustración 9: Ventana principal*

5.5. Cambiar la imagen de perfil

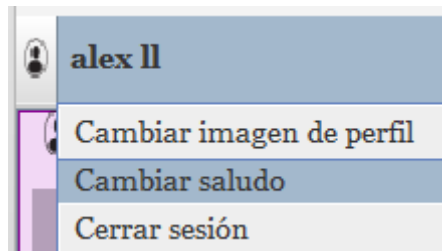
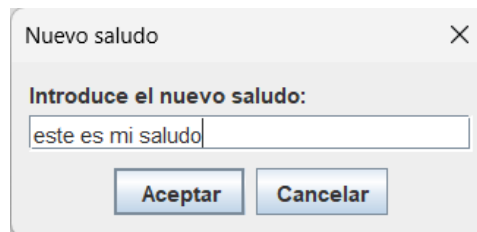
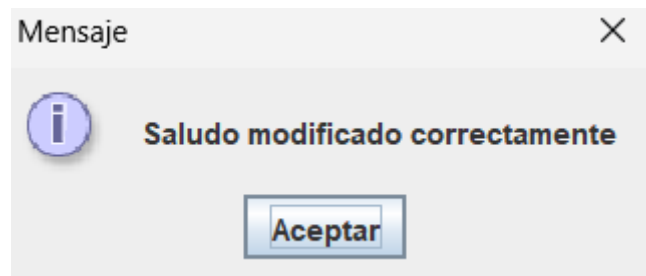
Para hacer el cambio de imagen de perfil del usuario se debe pinchar en el menú superior de la ventana principal y en el menú desplegable seleccionar la opción “Cambiar imagen de perfil”. Cuando se seleccione el botón saldrá la misma pestaña que la de seleccionar la imagen en el registro. En la pestaña se debe introducir la url de la nueva imagen de perfil y darle al botón aceptar.

*Ilustración 10: Darle a "cambiar imagen de perfil"**Ilustración 11: Introducir URL de imagen deseada*

*Ilustración 12: Notificación de imagen modificada**Ilustración 13: Imagen modificada*

5.6. Cambiar el saludo

El caso de cambiar el saludo es exactamente igual que la opción de cambiar la imagen de perfil, pero en el desplegable se selecciona la opción “Cambiar saludo”. En este caso también se abrirá una pestaña para que se introduzca la nueva presentación del usuario.

*Ilustración 14: Seleccionar "cambiar saludo"**Ilustración 15: Introduciendo saludo**Ilustración 16: Confirmación de saludo modificado*

5.7. Añadir contacto

Hay dos opciones de añadir un contacto y estas son:

1. A través de la agenda de contactos individuales:

En la Ilustración 9, encontramos que en el menú esta la opción de Gestión de Contactos si se selecciona esta opción se desplegará un menú y se deberá seleccionar la opción con el rótulo “Individuales”.

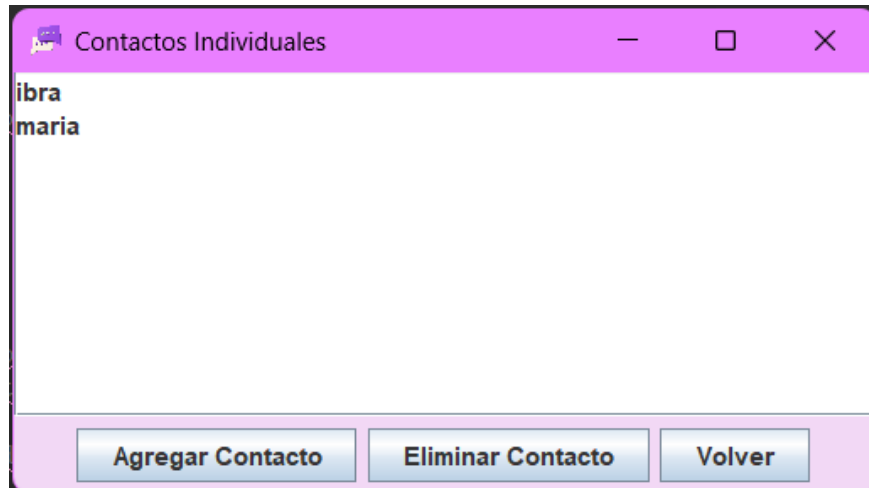


Ilustración 17: Ventana para gestionar los contactos individuales

Después se abrirá la ventana que tiene el registro de los contactos del usuario actual, la mostrada en la Ilustración 17. En esta ventana tenemos el botón “Agregar Contacto”, si lo presionamos se abrirá una pestaña para introducir el nombre que le queremos poner al nuevo contacto y su teléfono. Esta pestaña es la que se ilustra en la Ilustración 18. Si el móvil ya está agregado con otro contacto dará error, ya que no se puede agregar un número a dos contactos distintos.

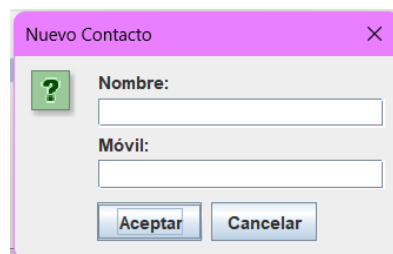


Ilustración 18: Pestaña para insertar los datos de un nuevo contacto

2. A través de la pestaña principal:

Si un usuario que no se tiene agregado manda un mensaje, en la ventana principal, en el listado de contactos encontraremos el contacto con un botón con el símbolo “+”. Si se presiona saldrá la misma pestaña que en el caso anterior pero esta vez con el campo Móvil ya completado con el número de teléfono del usuario que haya enviado el mensaje. Un ejemplo de cómo se vería se puede ver en la Ilustración 19.

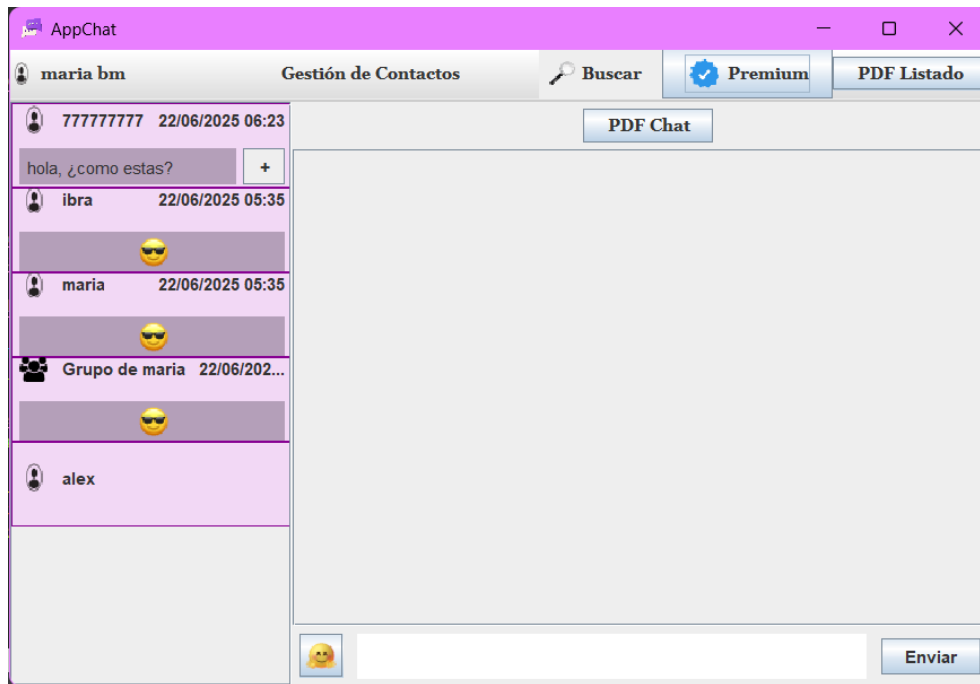


Ilustración 19: Ventana principal con ejemplo de contacto sin agregar

5.8. Eliminar contacto

En la misma ventana que se muestra en la Ilustración 17, encontramos el botón “Eliminar contacto”. Este botón mostrará un error si no se selecciona ningún contacto, pero si se selecciona un contacto preguntará la seguridad de la decisión; y si se acepta se borrará el contacto de la agenda del usuario.

5.9. Crear grupo

Para la gestión de los grupos también existe una ventana y esa es la mostrada en la Ilustración 20. En esta pestaña encontramos varios botones y entre ellos está el “Crear Grupo”. Este botón abre una pestaña la cual nos permitirá ponerle nombre a un grupo y al darle a “Aceptar” se creará el grupo. Cabe destacar que se crea sin miembros, que si queremos añadir miembros se deben añadir como se indica en el apartado *Añadir/Eliminar Miembro de un grupo*.

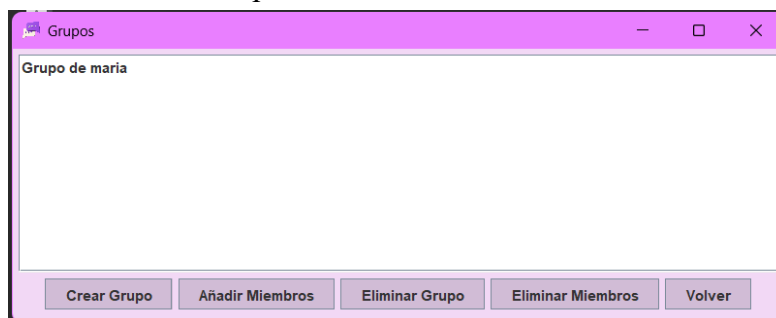


Ilustración 20: Ventana para gestionar los grupos

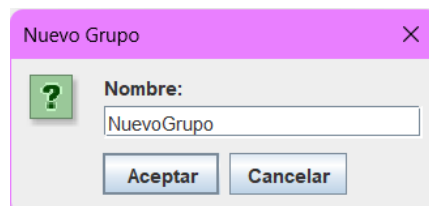


Ilustración 21: Pestaña nombre del grupo

5.10. Eliminar grupo

En la misma ventana que en el apartado anterior encontramos el botón “Eliminar Grupo”, que hace lo mismo que el botón “Eliminar Contacto” explicado en el apartado *Eliminar contacto*, pero extrapolado a los grupos.

5.11. Añadir/Eliminar Miembro de un grupo

Otra vez en la misma ventana, la que gestiona los grupos y se ve en la Ilustración 20, encontramos dos botones más:

- Añadir Miembros:

Si se presiona este botón se abrirá una ventana que nos permitirá elegir entre los contactos que tenemos agregados y todavía no pertenecen al grupo, cuales queremos añadir. Esta ventana se puede ver en la Ilustración 22.



Ilustración 22: Añadir miembros a un grupo

- Eliminar Miembros:

Si se presiona este botón se abrirá una ventana que nos permitirá elegir entre los contactos que pertenecen a ese grupo. Esta ventana se puede ver en la Ilustración 23.



Ilustración 23: Eliminar miembros de un grupo

Cabe destacar que tanto la Ilustración 22 y la Ilustración 23 se han realizado con el mismo grupo, por eso los que se ven seleccionados en la primera de estas son los que se añadieron y los únicos que se ven en la segunda.

Y, por último, al presionar uno de estos botones si no se ha seleccionado antes un grupo se mostrará un mensaje de error.

5.12. Crear los PDFs

Hay varios PDFs que se pueden crear, pero todas las opciones necesitan que el usuario sea un usuario premium. Si se desea convertir en usuario premium lea el apartado *Activación del premium*. Los diferentes PDFs se crean:

1. En el menú de la ventana mostrada en la Ilustración 9, encontramos un botón que se encarga de generar un archivo pdf que le puede servir al usuario como listado de los contactos y grupos que tiene agregados. Este botón se le puede distinguir por tener el rótulo “PDF Listado”. Un ejemplo del PDF que se genera es:

Contactos añadidos en AppChat de maria bm

¡Hola! Antes de nada, le agradecemos su máxima confianza depositada en AppChat. En primer lugar, he aquí un listado de sus contactos individuales, con sus respectivos números de teléfono móvil.

1. alex: 6666666666
2. ibra: 313131313
3. maria: 121212121

En segundo lugar, le mostramos un listado de sus grupos, con cada uno de los integrantes que añadió.

1. Grupo de maria:
 - ibra: 313131313
 - maria: 121212121
2. NuevoGrupo:
 - alex: 6666666666
 - maria: 121212121

Ilustración 24: Ejemplo PDF Listado generado

2. En el chat también de la misma ventana, en la parte alta encontramos el botón, pero esta vez con el rótulo “PDF Chat”. Este archivo solo se genera si hay algún chat de algún contacto o grupo abierto; sino se mostrará un error. Un ejemplo del PDF que se genera es:

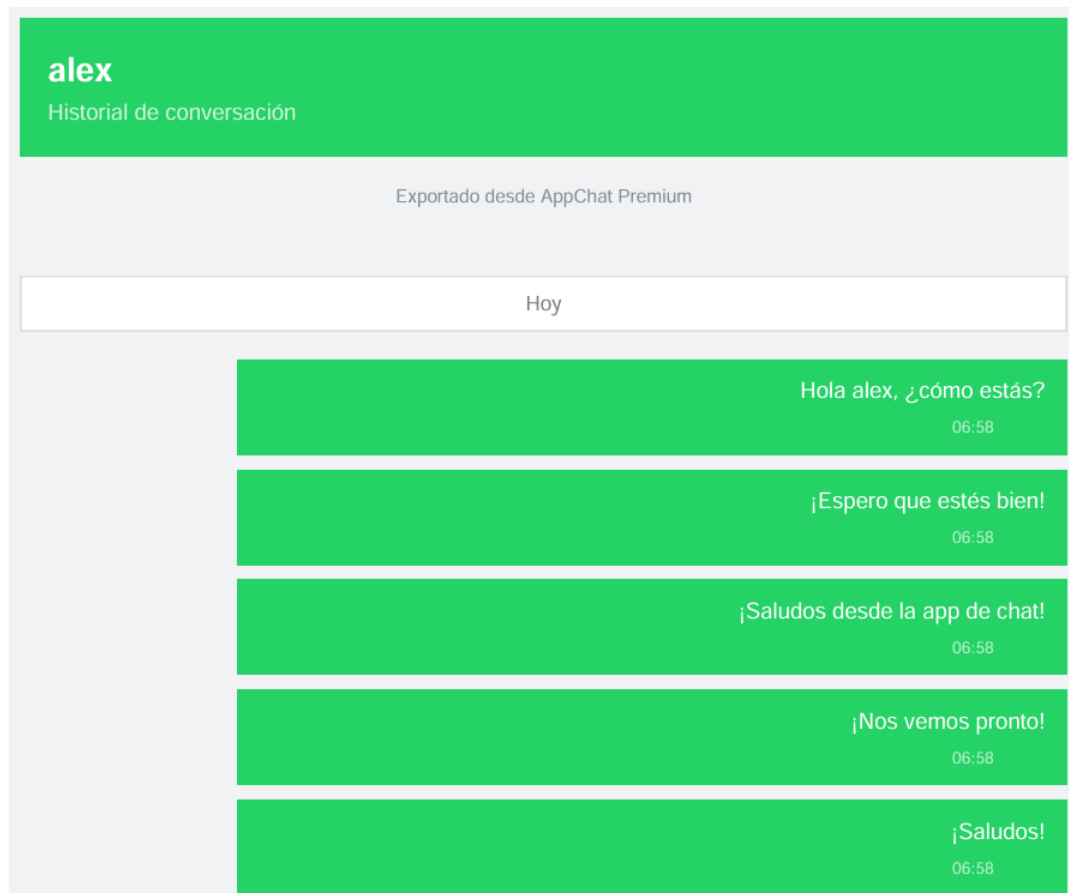


Ilustración 25: Ejemplo PDF Listado generado

***IMPORTANTE:** los emojis, al no tener texto, se muestran como mensajes vacíos.

5.13. Buscar mensajes

En la ventana principal encontramos, en el menú, el botón “Buscar”. Si es presionado se abrirá la ventana que se muestra en la Ilustración 26. En esta ventana encontramos 4 campos, de los 4 que podemos seleccionar.

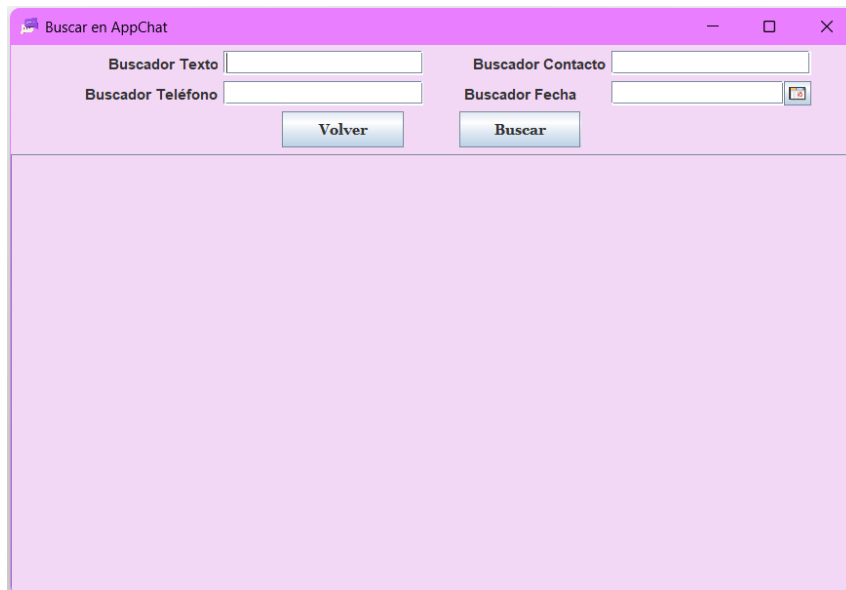


Ilustración 26: Ventana para buscar

Si no se completa ningún campo y se presiona el botón “Buscar” se mostrará un error. Al contrario, si se completa algún campo y se presiona el botón, se mostrará un listado con los mensajes que coincidan con algún filtro de los que se han activado. Aclaración: los filtros se activan solo con completar el campo, si un campo está vacío no se toma como activado. Un ejemplo de lo que se vería es lo que se ve en la Ilustración 27.

Si alguno de los mensajes que se muestran se presiona, entonces la aplicación abre el chat con ese contacto.

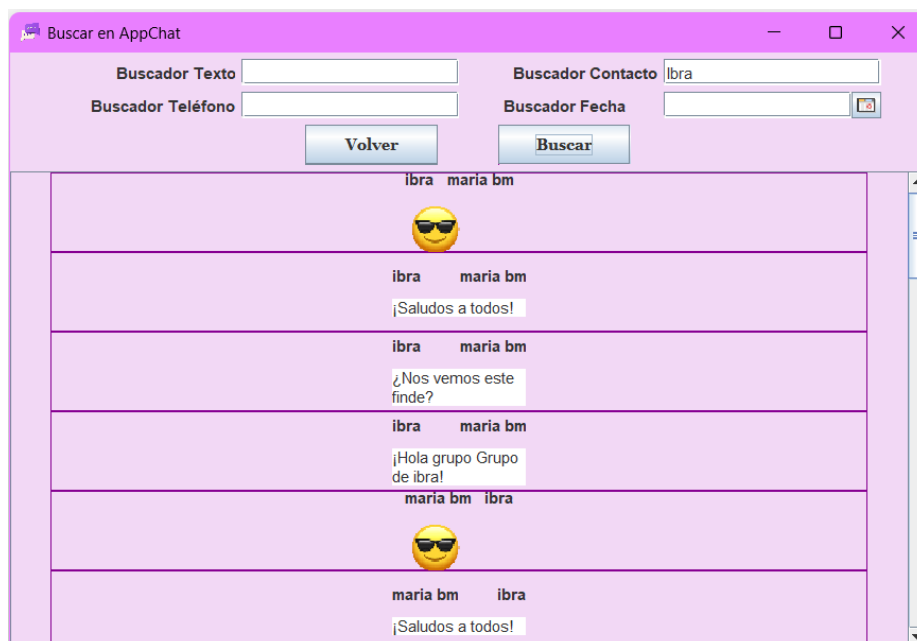


Ilustración 27: Ventana buscar funcionando

5.14. Activación del premium

En la ventana principal encontramos, en el menú, el botón “Premium”. Si ese botón es presionado se le preguntara al usuario si está seguro. Cuando acepte se le mostrará el importe y el tipo de descuento que se le aplica.

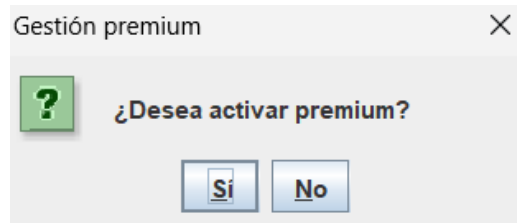


Ilustración 28: Pregunta por si confirmas activar premium

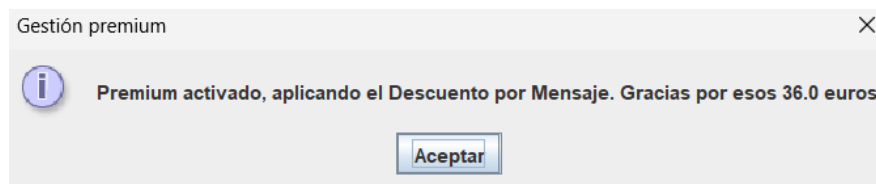


Ilustración 29: Notificación tras activar premium

6. Conclusiones

Conclusión María:

Considero que este ha sido el trabajo más largo y exigente al que me he enfrentado durante toda mi etapa educativa. A lo largo de los años he realizado numerosos proyectos, pero sin duda este ha sido el más complejo.

En esta asignatura tenemos un temario totalmente nuevo, con mucha información (hay muchos patrones de diseño) que tienes que dominar para conseguir hacer una mínima clase de este gran proyecto.

Aunque ya estaba acostumbrada al lenguaje de Java por anteriores asignaturas como POO o PCD, se nos ha introducido Swing. Esto tampoco es tan difícil si se entiende, el problema es que no hay forma de entenderlo sin practicarlo, pero al practicar te puedes llegar a desesperar (como cambiar un número en una coordenada x o y), ya que cualquier cambio podía afectar drásticamente la visualización en la pestaña Design. Yo para mejorar mi comprensión, recurrí a un curso en YouTube que me ayudó a controlar mejor el código de Swing y a generar componentes de forma dinámica, sin depender únicamente de la interfaz visual.

También ha sido la primera vez que trabajamos con un proyecto Maven, lo cual supuso una dificultad añadida. En mi caso, tuve problemas al instalar las dependencias y configurar el repositorio local en mi ordenador.

No valía con todo esto, que además hemos introducido el uso del GitHub. Cabe destacar que esta herramienta la veo fundamental en lo que me queda de etapa educativa y posteriormente en la laboral. Hasta ahora yo hacía los trabajos compartiendo una carpeta en la nube, pero tengo que admitir que ha sido mucho más fácil, eficiente y organizado con GitHub.

Otro reto importante fue el relacionado con el trabajo en equipo. Al principio, sentí que estaba enfrentándome sola a un proyecto demasiado extenso. Ante la falta de implicación de mi compañera inicial, decidí buscar un nuevo compañero, claramente con el consentimiento del profesor. Desde entonces, el trabajo, aunque ha sido difícil, se volvió más llevadero, ya que ambas partes nos implicamos por igual. Estoy convencida de que, si Ibrahim y yo hubiéramos trabajado juntos desde el principio, habríamos logrado entregar el proyecto en la convocatoria del primer cuatrimestre.

El tiempo que estimo que he tardado en hacer el proyecto de esta asignatura son entre 100 y 120 horas perfectamente, aunque podrían ser más. En esta aproximación he incluido las horas dedicadas con mi anterior compañera, ya que es trabajo que yo realice en las prácticas de esta asignatura.

Conclusión Ibrahim:

Creo que para mí este no ha sido un proyecto de programación más. Realmente ha sido un reto mayúsculo. Tanto por lo que es esta práctica en sí, como las circunstancias por las que he tenido que pasar.

En primer lugar, quisiera destacar lo realmente exigente que es este proyecto. Aquí ya no valía con saber programar algunas clases y ya está. Se nos introdujo a crear software de verdad, software con un propósito específico. Entonces claro, para crear un software, se requiere de una profunda comprensión de lo que es la arquitectura software, requisitos, el desarrollo iterativo e incremental, los patrones de diseño... Todo ello con el propósito de que el software que vayamos a crear sea lo más funcional, mantenible, y escalable posible. Son nuevos conceptos que considero que me han abierto la mente como desarrollador.

En segundo lugar, destaco también el aumento de dificultad a la hora de programar en Java. Se nos introdujo a Swing. En años anteriores, los programas que creábamos se ejecutaban desde

una consola. Obviamente así no funcionan las aplicaciones de verdad. Entonces para mí al principio supuso un choque, pero con el tiempo me pude ir adaptando. Además, otro aspecto nuevo es el de la persistencia de entidades. Antes los programas que creábamos y las entidades creadas existían únicamente durante el flujo de ejecución del programa. Pero ahora vi la forma con la cuál los servidores de persistencia nos facilitan el hecho de persistir entidades en una base de datos.

En tercer lugar, y esto ya es más personal, destacar lo imprescindible que es el trabajo en equipo. Durante el primer cuatrimestre iba con un compañero, pero se podría decir que estaba haciendo el proyecto solo. Entonces, ante la desesperación e impotencia de ver que no me daba para hacerlo solo, decidí encontrar nueva pareja. Y con mi compañera María, hemos sabido organizarnos la implementación del proyecto, y si bien nos hemos encontrado con supuestas adversidades, al final lo hemos sacado adelante como hemos podido. El mayor aprendizaje que me he llevado durante el transcurso del desarrollo, es que en el desarrollo de software es crucial contar con un entorno colaborativo.

En definitiva, el desarrollo de AppChat ha supuesto un desafío mayúsculo, pero creo que ha merecido la pena. Supone un punto de inflexión, el cual espero que haya sido positivo y suponga un paso adelante en mi corta experiencia programando.

PD: Estimo un mínimo de 120 horas de trabajo, los cuales la inmensa mayoría ya han sido trabajando con mi compañera actual.